PROJECT SYNOPSIS
“AGILEGRAPH”

Graph-Learned Crypto-Agility Risk Scoring for Post-Quantum Migration
(Heterogeneous GNN Risk Propagation and Explainable Migration Prioritization)

Students Names & USN:
1. Spoorthi — NNM23CB058
2. Sindhoora R — NNM23CB057
3. Nivedita V Shetty — NNM23CB033
4. Sujan S — NNM23CB062

Guide Name: Ms. Madhushree
Department: Computer Science & Engineering (Cyber Security)
Institution: NMAM Institute of Technology (NMAMIT), Nitte
Academic Year: 2023 – 2027 (Final Year, B.Tech)

1. Introduction

The advent of Cryptanalytically Relevant Quantum Computers (CRQCs) presents an existential threat to modern asymmetric cryptography. Shor’s algorithm shows that a cryptanalytically relevant quantum computer operating in polynomial time can compromise public-key cryptosystems based on integer factorization (RSA) and discrete logarithms over finite fields and elliptic curves (DSA, ECDSA, ECDH). In response, the National Institute of Standards and Technology (NIST) finalized the primary post-quantum cryptographic standards in August 2024: FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), and FIPS 205 (SLH-DSA).

However, transitioning modern polyglot enterprise software architectures to quantum-resistant primitives is an arduous systems engineering challenge. Organizations face the critical threat of **Harvest Now, Decrypt Later (HNDL)** attacks, where adversarial entities intercept and store encrypted sensitive data today to decrypt once quantum machines emerge. Because modern software is deeply modular and dependent on third-party supply chains, cryptographic primitives are dispersed across application source code, containerized endpoints, external package manifests, digital certificates, and continuous integration pipelines.

AgileGraph is proposed as an automated cryptographic discovery, typed property-graph modeling, and crypto-agility risk scoring system. Rather than treating codebases as flat lists of dependencies, AgileGraph structures polyglot source code (Python, Java, Go), dependency manifests, and infrastructure endpoints into a formal **6-node typed property graph**. It evaluates migration urgency through Michele Mosca’s Theorem ($U = (x+y)/z$), computes deterministic 7-factor heuristic risk scores with missing-context handling, tracks asset remediation across a 6-state lifecycle roadmap, and investigates a Heterogeneous Graph Attention Network (GATv2) for topological risk propagation and representation distillation.

2. Problem Statement

Current software supply chain and cryptographic inventory approaches suffer from four fundamental limitations:

1. Flat, Disconnected Inventories: Conventional Software Bill of Materials (SBOM) and Software Composition Analysis (SCA) tools treat software as flat dependency manifests, failing to capture architectural call paths, data flow exposure, or the structural centrality of cryptographic primitives.

2. Ignorance of Post-Quantum Urgency: Standard vulnerability scanners evaluate severity based on historical Common Vulnerability Scoring System (CVSS) metrics rather than forward-looking quantum threat timelines and data confidentiality retention requirements (Mosca's Theorem).

3. Rigid, Monolithic Scoring Without Context Handling: Existing rule engines fail when operational context (e.g., internet exposure or dynamic data classification) cannot be statically observed, either defaulting to zero risk (dangerous underestimation) or failing completely.

4. Lack of Lifecycle Remediation Tracking: Security scanners report point-in-time vulnerability lists without supporting the multi-year, phased cryptographic agility migration lifecycle (from discovery through dual-mode hybrid testing to full algorithm decommissioning).

3. Objectives

1. **To Design a Canonical Property Graph Schema:** Formalize a 6-node property-graph ontology (*File, CryptoUsage, Certificate, Endpoint, Library, SensitiveData*) and 11 typed directed edge relations (*CALLS, USES_LIBRARY, PROTECTS, EXPOSES, CONTAINS, DEPENDS_ON*, and inverse message-passing counterparts) with strict topological invariant verification.
2. **To Develop Multi-Language Static Scanners:** Build a deterministic static discovery engine comprising 7 specialized scanners across 3 programming languages (Python AST, Java JCA/JCE, Go) and environmental analyzers (Dependency manifests with offline OSV CVE intelligence, X.509 Certificate parser, Endpoint scanner, and Sensitive Data detector).
3. **To Implement Deterministic 7-Factor Risk Scoring & Mosca Urgency:** Formulate a transparent, normalized composite risk engine ($R(v) \in [0.0, 1.0]$) with canonical weights summing to 1.0, missing-evidence handling policies (*RENORMALIZE, CONSERVATIVE*), and the Mosca Readiness Index ($U = (X+Y)/Z$, $X+Y > Z$) for forward-looking quantum migration urgency.
4. **To Evaluate Heterogeneous GATv2 Representation Distillation:** Implement and evaluate a PyTorch-Geometric Heterogeneous GATv2 model trained on weak labels from the heuristic engine under Leave-One-Project-Out (LOPO) cross-validation across an 8-repository corpus, verified against edge-shuffled topological controls.
5. **To Integrate Graph Explainability & Independent Expert Validation:** Incorporate PyTorch Geometric's GNNExplainer to extract local feature saliency and influential topological edges for individual risk predictions, and execute an independent 4-expert validation protocol across 160 repository-scoped assets evaluated via Weighted Cohen's Kappa, Fleiss' Kappa, and bootstrap confidence intervals without circular training leakage.
6. **To Build & Validate an Enterprise Product Platform:** Deliver a production-grade web dashboard (React 19, Vite, TypeScript) and REST API (FastAPI) backed by an append-only SQLite audit log tracking assets across a 6-state migration roadmap (*DISCOVERED* → *DEPRECATED*), multi-user isolation, and SHA-256 provenance CBOM reports.

4. Literature Survey

4.1 Existing System

Existing enterprise crypto-inventory mechanisms typically rely on basic static analysis (SAST) linters (e.g., Bandit for Python, SpotBugs/FindSecBugs for Java) or commercial SCA tools (e.g., Snyk, Mend). These tools execute regex-based or token-based searches to flag deprecated algorithm names (such as MD5, SHA-1, or DES). Recently, cryptographic bill of materials (CBOM) standards (e.g., OWASP CycloneDX) have emerged to catalog cryptographic assets such as algorithms, keys, certificates, and protocols across software components.

4.2 Limitations of Existing System

Existing systems fail to model the relational architecture connecting code files to endpoints and libraries. A deprecated cipher buried in an offline test suite is given the same severity rating as the same cipher securing a public API endpoint. Furthermore, most existing tools do not incorporate Michele Mosca's post-quantum urgency timeline, making it difficult for engineering teams to prioritize migration based on data retention requirements.

4.3 Related Research Papers

Sl.	Paper Title & Venue	Authors & Year	Key Contribution	Gap Addressed by AgileGraph
1	Cybersecurity in an Era with Quantum Computers: Will We Be Ready? (IEEE Security & Privacy)	M. Mosca (2018)	Formulated Mosca’s inequality (x+y > z) defining quantum migration deadlines and shelf-life urgency.	Lacks an automated static analysis engine to extract x, y, and code-level vulnerability parameters.
2	Migration to Post-Quantum Cryptography: Cryptographic Discovery (NIST SP 1800-38B, Preliminary Draft)	NIST NCCoE (2023)	Outlined architectural guidelines and discovery requirements for finding quantum-vulnerable cryptography.	Defines organizational guidance without providing a formal typed property graph model or structural risk engine.
3	An Empirical Study of Cryptographic Misuse in Android Applications (ACM CCS)	M. Egele et al. (2013)	Demonstrated automated static analysis rules for detecting cryptographic API misuse across Android applications.	Focuses on classical API misuses in Android apps without evaluating post-quantum threat horizons or multi-layer dependency graph properties across enterprise polyglot codebases.
4	How Attentive Are Graph Attention Networks? (ICLR)	S. Brody et al. (2022)	Proved dynamic attention limitations in standard GAT and introduced GATv2 with strictly dynamic attention.	Evaluated on general graph benchmarks, not on cryptographic risk propagation graphs.
5	CogniCrypt: Supporting Developers in using Cryptography (IEEE/ACM ASE)	S. Krüger et al. (2017)	Integrated continuous static analysis into developer workflows to detect cryptographic API misuse rules.	Restricted to single-language (Java) rule checking without post-quantum urgency scoring or multi-layer dependency graph triage.

5. Proposed System

The proposed AgileGraph architecture operates through seven cohesive technical layers:

1. Comprehensive 7-Scanner Discovery Suite: Executes 7 specialized static scanners mapping directly into the 6 property-graph node categories: (a) Python AST Scanner (hashlib, hmac, cryptography.hazmat, PyCryptodome), (b) Java Scanner (JCA/JCE Cipher, KeyPairGenerator, jjwt, java-jwt), (c) Go Scanner (crypto/*, golang-jwt), (d) Dependency Manifest Scanner (requirements.txt, pom.xml, go.mod with local OSV CVE cache), (e) Certificate & Key Scanner (X.509 RFC 5280 parsing, inline PEM blocks, private keys), (f) Network Endpoint Scanner (service bindings, port configurations, route decorators), and (g) Sensitive Data Scanner (Shannon entropy calculation, secret tokens, private credentials).

2. 6-Node Typed Property Graph: Models software architecture in NetworkX across *File, CryptoUsage, Certificate, Endpoint, Library,* and *SensitiveData* nodes interconnected by 11 typed directed edge relations (*CALLS, USES_LIBRARY, PROTECTS, EXPOSES, CONTAINS, DEPENDS_ON*, and inverse message-passing counterparts).

3. Deterministic 7-Factor Risk Engine: Computes normalized risk scores $R(v) \in [0.0, 1.0]$ based on algorithmic weakness (NIST PQC status), asset criticality, data sensitivity, internet exposure, migration difficulty, library centrality, and CVE severity, with robust missing-evidence handling (*RENORMALIZE* and *CONSERVATIVE*).

4. Mosca Urgency & Triage Matrix: Computes both the inequality condition $X + Y > Z$ and normalized urgency index $U = (X+Y)/Z$ to categorize assets into *AT_RISK, URGENT,* and *SAFE,* triaging vulnerabilities into *CRITICAL, HIGH, MEDIUM,* and *LOW* priority bands.

5. Heterogeneous GATv2 Neural Network (Representation Distillation): Implements a PyG HeteroGATv2 model trained on weak labels derived from the deterministic heuristic risk engine to evaluate topological representation distillation under Leave-One-Project-Out cross-validation (empirically evaluating weak-label distillation across domain shift rather than claiming independent prediction of unverified real-world cryptanalytic outcomes).

6. Graph Explainability & Independent Expert Validation: Integrates PyTorch Geometric's GNNExplainer to decompose GATv2 risk scores into local feature attributions and influential relational edges, paired with an independent 4-expert panel validation protocol across 160 repository-scoped source-code assets. Recalculated metrics independently verify: high multi-rater panel concordance (Fleiss' Kappa = 0.7791 on N = 154 mutually rated assets with 6 Expert 4 abstentions due to insufficient context; mean pairwise linear weighted Cohen's Kappa = 0.8167 across all 6 pairs). Consensus adjudication via majority voting with conservative tie-breaking (HIGH > MEDIUM > LOW) established 80 HIGH, 51 MEDIUM, and 29 LOW ground-truth assets. Evaluation of the 7-factor heuristic engine against expert consensus yielded Macro-F1 = 0.3225 with a 95% bootstrap confidence interval of [0.2558, 0.3799] (B = 1000) and weighted Cohen's Kappa = 0.2625, highlighting that while experts evaluate intrinsic cryptographic exploitability (Shor/Grover vulnerability of public-key ciphers), AgileGraph's heuristic engine incorporates exposure and graph reachability to temper risk on non-production test fixtures and library wrappers. Strict data segregation guarantees zero expert label leakage into GNN model weights.

7. Presentation & Lifecycle: An interactive React 19/TypeScript web dashboard featuring 2D force-directed graph rendering, an explainability drawer, a migration roadmap view tracking assets across 6 remediation states, and verifiable audit report exports carrying Git commit SHA and heuristic configuration SHA-256 verification hashes.

6. System Requirements

6.1 Software Requirements

• **Operating System:** Ubuntu Linux 22.04 LTS / Windows 11 / macOS Sonoma
• **Programming Languages:** Python 3.10+ (Backend & GNN), TypeScript 5.0+ (Frontend)
• **Frameworks:** FastAPI, Uvicorn, Pydantic v2, React 19, Vite, TailwindCSS v4
• **Libraries:** NetworkX 3.2+, PyTorch 2.4+, PyTorch Geometric (PyG) 2.6+, Scikit-Learn, SciPy
• **Database:** SQLite (relational storage for session security and append-only lifecycle audit trail)
• **Security:** bcrypt (12 rounds) + PyJWT (HS256) bearer authentication
• **Packaging:** Dockerfile (containerized CLI runner)

6.2 Hardware Requirements

• **Processor:** Quad-Core Intel Core i5 / AMD Ryzen 5 or higher (Apple Silicon supported)
• **RAM:** 8 GB minimum (16 GB recommended for GATv2 model training)
• **Storage:** 10 GB available SSD space

7. Expected Outcomes

1. **Automated Polyglot 7-Scanner Suite:** A deterministic CLI and FastAPI REST service executing 7 specialized scanners across Python, Java, Go, package manifests, certificates, endpoints, and secrets.
2. **Interactive Web Dashboard:** A full-stack application featuring 2D force-directed graph visualization, inventory ledgers, GNNExplainer attribution drawer, and Mosca calculators.
3. **Migration Roadmap Lifecycle Workflow:** Lifecycle tracking enabling engineering teams to transition assets from DISCOVERED to DECOMMISSIONED with an append-only SQLite audit log.
4. **Reproducible LOPO Benchmark Study:** Empirical cross-validation across 8 benchmark repositories (520 files) documenting representation distillation.
5. **GNNExplainer Attribution & Expert Validation Protocol:** Graph-based local feature/edge explanations combined with an independent 4-expert validation protocol across 160 repository-scoped assets (Fleiss' Kappa = 0.7791 on N = 154 mutually rated assets, mean pairwise linear weighted Cohen's Kappa = 0.8167, Heuristic Macro-F1 = 0.3225, 95% CI: [0.2558, 0.3799]) with complete data segregation from neural weights.
6. **Verifiable Audit Exporters:** Export of compliance documentation in JSON and Markdown formats carrying Git commit SHA and heuristic configuration SHA-256 verification hashes.

8. Applications

- **Enterprise Post-Quantum Migration:** Enables CISOs and security architects to identify and prioritize high-risk cryptographic dependencies before quantum hardware scales.
- **DevSecOps & CI/CD Gating:** Integrates into continuous deployment pipelines to prevent developers from introducing non-agile, quantum-vulnerable cryptographic algorithms into production.
- **Cybersecurity Compliance Auditing:** Automatically generates verifiable cryptographic compliance documentation aligned with NIST FIPS 203/204/205, BSI post-quantum roadmaps, and NSA CNSA 2.0 timelines.
- **Software Supply Chain Risk Management:** Evaluates open-source libraries and proprietary vendor packages to assess transitive cryptographic exposure and structural library centrality.

9. Future Scope

- **Automated Code Patching via LLMs:** Integrating fine-tuned Large Language Models to automatically generate code pull requests that refactor deprecated cryptographic calls into quantum-safe primitives.
- **Runtime Passive eBPF Agent:** Expanding discovery from static code analysis to kernel-level eBPF probes that monitor live TLS handshakes, dynamic certificate rotations, and active socket connections in production Kubernetes clusters.
- **Distributed Graph Database Migration:** Upgrading the in-memory NetworkX storage to distributed graph databases (such as Memgraph or Apache AGE) to support enterprise code repositories exceeding millions of source files.

10. Project Timeline

Phase	Milestone / Activity Description	Duration	Target Deliverable
Phase 1	Requirement analysis, NIST PQC review, AST scanner development for Python, Java, Go.	Weeks 1–4	Multi-language AST discovery modules and reference test fixtures.
Phase 2	Graph schema formulation, 6-node ontology design, and NetworkX property graph builder.	Weeks 5–8	Validated property graph generator producing verified edge triplets.
Phase 3	Deterministic 7-factor risk engine, missing data policies, and Mosca theorem urgency integration.	Weeks 9–12	Verified mathematical scoring engine and JSON export utilities.
Phase 4	Benchmark corpus compilation (8 repos, 520 files), GATv2 model engineering, and LOPO cross-validation.	Weeks 13–16	Evaluated PyG GNN pipeline, multi-seed stability results, and baseline comparisons.
Phase 5	Protected React 19 web dashboard, Migration Roadmap lifecycle view, JWT authentication, and CLI reporting.	Weeks 17–20	Fully integrated full-stack web application with force-directed graph rendering.
Phase 6	System hardening, Docker container packaging, comprehensive regression testing, documentation, and final viva defense.	Weeks 21–24	Final project report, research paper manuscript, and production-ready source code.

11. References

[1] National Institute of Standards and Technology (NIST), “Module-Lattice-Based Key-Encapsulation Mechanism Standard,” Federal Information Processing Standards Publication (FIPS PUB) 203, Aug. 2024.

[2] National Institute of Standards and Technology (NIST), “Module-Lattice-Based Digital Signature Standard,” Federal Information Processing Standards Publication (FIPS PUB) 204, Aug. 2024.

[3] National Institute of Standards and Technology (NIST), “Stateless Hash-Based Digital Signature Standard,” Federal Information Processing Standards Publication (FIPS PUB) 205, Aug. 2024.

[4] M. Mosca, “Cybersecurity in an Era with Quantum Computers: Will We Be Ready?,” IEEE Security & Privacy, vol. 16, no. 5, pp. 38–41, Sep./Oct. 2018, doi: 10.1109/MSP.2018.3761723. (Also: IACR Cryptology ePrint Archive, Report 2015/1075, 2015).

[5] P. W. Shor, “Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer,” SIAM Journal on Computing, vol. 26, no. 5, pp. 1484–1509, Oct. 1997, doi: 10.1137/S0097539795293172.

[6] S. Brody, U. Alon, and E. Yahav, “How Attentive Are Graph Attention Networks?,” in Proc. 10th Int. Conf. Learn. Represent. (ICLR), Virtual Event, Apr. 2022, pp. 1–18.

[7] P. Veli■kovi■, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph Attention Networks,” in Proc. 6th Int. Conf. Learn. Represent. (ICLR), Vancouver, Canada, Apr. 2018, pp. 1–12.

[8] National Institute of Standards and Technology (NIST), “Migration to Post-Quantum Cryptography: Quantum Readiness: Cryptographic Discovery,” NIST Special Publication (SP) 1800-38B (Preliminary Draft), NCCoE, Dec. 2023.

[9] OWASP Foundation, “CycloneDX v1.6 Released, Advances Software Supply Chain Security with Cryptographic Bill of Materials and Attestations,” OWASP CycloneDX News, Apr. 2024. [Online]. Available: https://cyclonedx.org/news/cyclonedx-v1.6-released/

[10] M. Egele, D. Brumley, Y. Fratantonio, and C. Kruegel, “An Empirical Study of Cryptographic Misuse in Android Applications,” in Proc. 2013 ACM SIGSAC Conf. Comput. Commun. Secur. (CCS), Berlin, Germany, Nov. 2013, pp. 73–84, doi: 10.1145/2508859.2516693.

[11] S. Krüger, J. Späth, K. Ali, E. Bodden, and M. Mezini, “CogniCrypt: Supporting Developers in using Cryptography,” in Proc. 32nd IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE), Urbana, IL, USA, Nov. 2017, pp. 931–936, doi: 10.1109/ASE.2017.8115707.

[12] National Security Agency (NSA), “Commercial National Security Algorithm Suite 2.0 (CNSA 2.0) Cybersecurity Advisory,” NSA Cybersecurity Technical Report, Sep. 2022.

[13] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, “Modeling and Discovering Vulnerabilities with Code Property Graphs,” in Proc. 2014 IEEE Symp. Secur. Privacy (S&P), San Jose, CA, USA, May 2014, pp. 590–604, doi: 10.1109/SP.2014.44.

[14] B. Sullivan, “Cryptographic Agility,” IEEE Security & Privacy, vol. 11, no. 3, pp. 73–75, May/Jun. 2013, doi: 10.1109/MSP.2013.72.

[15] M. Fey and J. E. Lenssen, “Fast Graph Representation Learning with PyTorch Geometric,” in ICLR Workshop on Representation Learning on Graphs and Manifolds, New Orleans, LA, USA, May 2019, pp. 1–9.

[16] P. Schwabe, D. Stebila, and T. Wiggers, “Post-Quantum TLS Without Handshake Signatures,” in Proc. 2020 ACM SIGSAC Conf. Comput. Commun. Secur. (CCS), Virtual Event, Nov. 2020, pp. 1461–1480, doi: 10.1145/3372297.3417884.

[17] P. Ladisa, H. Plate, M. Martinez, and O. Barais, “SoK: Taxonomy of Attacks on Open-Source Software Supply Chains,” in Proc. 2023 IEEE Symp. Secur. Privacy (S&P), San Francisco, CA, USA, May 2023, pp. 1509–1526, doi: 10.1109/SP46215.2023.10179304.

[18] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, “Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 32, Vancouver, Canada, Dec. 2019, pp. 10197–10207.

[19] D. Moody, “The Transition to Post-Quantum Cryptography,” IEEE Security & Privacy, vol. 21, no. 5, pp. 4–6, Sep./Oct. 2023, doi: 10.1109/MSEC.2023.3297138.

[20] D. A. Basin, C. J. F. Cremers, and C. Meadows, “Model Checking Security Protocols,” in Handbook of Model Checking, E. M. Clarke et al., Eds., Cham: Springer, 2018, pp. 727–762, doi: 10.1007/978-3-319-10575-8_22.

Student Signatures:

1. Spoorthi (NNM23CB058): _____

2. Sindhoora R (NNM23CB057): _____

3. Nivedita V Shetty (NNM23CB033): _____

4. Sujan S (NNM23CB062): _____

Guide Signature: _____
(Ms. Madhushree)
Department of Computer Science and Engineering (Cyber Security)
NMAMIT, Nitte